

Cours 420-243-LI Développement informatique Hiver 2026 Cégep Limoilou Département d'informatique Professeurs : Julien Brunet, Matheu Bourgoin	Travail pratique 3 Développement de tests unitaires 16% de la session
--	---

Objectifs

- Concevoir des tests unitaires
- Concevoir des validations de méthodes
- Concevoir la documentation quant aux contrats
- Concevoir un plan de test

À remettre

- Le travail sera remis sur Léa à la date indiquée.

Contexte :

- **Le travail se fait seul**
- ✓ **On veut reprendre un logiciel déjà existant et le solidifier en ajoutant des validations et des tests unitaires. On va procéder en 5 parties :**
 1. Ajouter des validations 30 %
 2. Documenter, selon la théorie des contrats, vos validations 20%
 3. Créer des tests unitaires qui couvrent bien l'ensemble des scénarios possibles 40%
- ✓ Avoir un plan de test cohérent 5%
- ✓ Utilisation adéquate de GitHub 5%

À faire

Utilisez le fichier « TP3 - vos initiales.iml » qui vous a été fourni dans le dossier « TP3 - Étudiant ». Renommez ce fichier en remplaçant les initiales par les vôtres. Vous y trouverez également le fichier « Énoncé programme.pdf », qui présente le programme déjà développé; vous pourrez vous y référer pour mieux en comprendre le fonctionnement.

Attention : il est important que le dossier *src* soit à la racine du projet, puisque le chemin d'accès aux fichiers en dépend.

1. Créer un nouveau dépôt distant et en faire l'utilisation lors du développement du TP3. On doit y voir des commits fréquents avec des messages pertinents
2. Dans le projet IntelliJ, faire la création d'un package *tests*
3. Pour chaque classe dans logique, excluant « Program », vous devez faire la création d'une classe de tests correspondant dans le package tests
4. Ajouter les validations nécessaires dans les classes contenu dans le package *logique*
 - Voir la section *Modification des classes existantes en fonction des requis*
5. Documenter le point 3 en s'appuyant sur les standards vus en classe.
6. Écrire les tests unitaires; Écrire des tests, pour **chacun des cas limites** ainsi **que les cas légitimes**, pour chacune des méthodes. Prenez soin de mettre en place le `@BeforeEach` et le `@AfterEach` si applicable.
7. Inscrire, en commentaire dans le haut de la classe, l'ordre de conception des tests pour chacune des classes de tests.

```
/**  
 * Author : Mathieu Bourgoïn  
 * Ordre de conception : 3e  
 */
```

Modification des classes existantes en fonction des requis

Spécifications pour la classe **Alchimiste**

- a) Le nom ne peut pas être null et doit contenir minimalement six caractères
- b) La méthode fairePotion
 - i) Recette ne peut pas être null

Spécifications pour la classe **Laboratoire**

Vous devez déduire en fonction de votre compréhension du programme les valeurs permises et ajuster les méthodes en conséquence pour y ajouter des validations ainsi que documenter le tout. N'hésitez pas à valider vos hypothèses avec votre enseignant.

Spécifications pour la classe **Ingredient**

- a) Le nom ne peut pas être null et doit contenir minimalement six caractères
- b) Le prix doit être supérieur à 0

Spécifications pour la classe **ResultatExperience**

Aucune spécification

Spécifications pour la classe **Recette**

- a) Aucun des ingrédients ne peut être null
- b) Il n'est pas possible d'avoir deux fois le même ingrédient dans une recette
- c) Le nom ne peut pas être null et doit contenir minimalement dix caractères
- d) La difficulté doit être entre les valeurs 1 et 5 inclusivement
- e) Le nombre de points d'expérience doit être strictement plus grand que 0
- f) La méthode contientIngredient
 - i) nom ne peut pas être null

Remarque :

La méthode fairePotion() de l'Alchimiste a un comportement aléatoire, il est donc délicat de tester la réussite ou non d'une potion. Pour simplifier le problème on vous propose d'identifier deux cas de tests seulement:

- Un cas où la réussite est garantie
- Un cas où l'échec est garanti

Critères d'évaluation

1. Les consignes sont respectées et le travail est complet.
2. Tous les tests s'exécutent avec succès
3. Le code final compile et il est clair
4. Les tests sont pertinents
5. La documentation est claire et explicite
6. Tous les paramètres et tous les attributs ont des types adéquats.
7. Tous les noms de classes et d'attributs respectent les standards Java.
8. Les consignes de mise en forme sont respectées

Dépôt

Il faut déposer sur LEA un zip contenant votre projet java